

Abschlussbericht: Analyse von IDE-Schnittstellen und Konzept einer Frontend-UI für Code-Agenten

1. Einleitung

Dieser Bericht adressiert die Nutzeranfrage zur Machbarkeit der Entwicklung eines Frontend-Tools, das Code-Agenten über verschiedene Integrierte Entwicklungsumgebungen (IDEs) hinweg steuern und einen gemeinsamen, synchronisierten Arbeitskontext ('Thread') für diese Agenten bereitstellen kann. Basierend auf der Analyse der Kommunikationsmechanismen von Stagewise und der Untersuchung der externen Schnittstellen von Cursor und Windsurf wird ein Konzept für eine solche Frontend-UI vorgestellt und die technische Realisierbarkeit, insbesondere im Hinblick auf die Integration der genannten IDEs, bewertet.

2. Analyse der Stagewise-Interaktionsmechanismen

Die Analyse der Stagewise-Architektur, insbesondere der Frontend-Komponente (Toolbar), zeigt einen klaren Ansatz zur Interaktion mit lokalen IDE-Agenten oder Servern.

- **Primärer Kommunikationskanal:** Stagewise nutzt **WebSockets** und ein **Structured RPC (SRPC)**-Protokoll für den bidirektionalen Datenaustausch. Dies ermöglicht eine persistente Verbindung und den Austausch strukturierter Befehle und Daten zwischen der Frontend-UI und der Backend-Logik, die idealerweise in der Nähe der IDE läuft (z.B. als lokaler Agent oder eine IDE-Erweiterung).
- **Kontexterfassung:** Plugins innerhalb der Stagewise-Umgebung sind für die Sammlung von Kontextinformationen (Code-Snippets, Dateipfade etc.) zuständig. Diese kontextuellen Daten werden zusammen mit UI-Kontext und Nutzereingaben zu Prompts aggregiert und über SRPC an die Code-Agenten übermittelt.
- **IDE-Erkennung:** Es gibt Hinweise darauf, dass Stagewise versucht, laufende VS Code-Fenster durch Mechanismen wie Port-Scanning zu erkennen und Verbindungen aufzubauen. Dies deutet auf eine erwartete, VS Code-kompatible Endpunkt-Schnittstelle hin.

Zusammenfassend ist die Stagewise-Architektur darauf ausgelegt, über standardisierte, netzwerkbasierte Protokolle (WS/SRPC) mit einer Remote- oder lokalen Komponente zu kommunizieren, die den Zugriff auf die IDE und die Ausführung von Agenten-Befehlen ermöglicht.

3. Externe Steuerungsmöglichkeiten für IDEs

Die Untersuchung der Schnittstellen der Ziel-IDEs (Cursor und Windsurf) ergab signifikante Unterschiede in ihren Möglichkeiten zur externen Steuerung.

- **Cursor:**

- Basierend auf VS Code erbt Cursor potenziell die Unterstützung für die **VS Code-Erweiterungs-API**, die **Befehlspalette** und die Möglichkeit zum Starten über die **Kommandozeile** (`cursor .`).
- Die Recherche ergab jedoch, dass es **keine öffentlich dokumentierte, dedizierte API oder CLI für die direkte programmatische Steuerung der Cursor IDE von extern** gibt.
- Die Analyse der Stagewise `srpc.ts` deutet stark darauf hin, dass eine Verbindung zu einem VS Code-kompatiblen Endpunkt (möglicherweise über eine Stagewise-eigene Erweiterung in Cursor) die vorgesehene Methode für Stagewise wäre, um über WS/SRPC zu kommunizieren.

- **Windsurf:**

- Für Windsurf IDE (ehemals Codeium Editor) wurde festgestellt, dass **keine öffentliche API, keine CLI für externe Steuerung, kein SDK zur Plugin-Entwicklung oder ein anderer dokumentierter Mechanismus für die externe programmatische Steuerung existiert**.
- Das existierende Model Context Protocol (MCP) dient lediglich dazu, *externe Daten und Kontext an den internen AI-Agenten* zu liefern, nicht dazu, die IDE selbst zu steuern.
- Windsurf unterstützt **keine VS Code Marketplace-Erweiterungen**, was eine Integration über diesen gängigen Pfad ausschließt.

Schlussfolgerung der Schnittstellenanalyse: Während Cursor (aufgrund seiner VS Code-Basis) potenziell eine Integration über eine spezifische Erweiterung ermöglicht, die eine kompatible Schnittstelle bereitstellt, ist eine externe Steuerung von Windsurf über dokumentierte und zuverlässige Kanäle derzeit **nicht möglich**.

4. Konzept für eine Frontend-UI zur Steuerung von Code-Agenten

Basierend auf diesen Erkenntnissen wird ein Konzept für eine Frontend-UI vorgeschlagen, die als zentraler Hub für die Interaktion mit Code-Agenten dient und einen gemeinsamen Kontext verwaltet.

4.1. Architektur der UI und Interaktion

Die UI fungiert als zentrale Steuereinheit. Sie kommuniziert primär mit unterstützten IDEs (wie Cursor, sofern eine passende Integration besteht) über die identifizierten Kanäle (WebSockets/SRPC).

Konzeptionelle Darstellung der Frontend-UI Architektur.

Die UI hält den globalen Zustand des "Shared Threads" (gemeinsamer Kontext) und sendet Befehle an die verbundenen Agenten, angereichert mit relevanten Kontextinformationen. Antworten und Status-Updates der Agenten werden von den IDEs zurück an die UI gesendet und im Kontext sowie einem Aktivitäts-Feed visualisiert.

4.2. Mechanismen zur Anweisungssendung

Die Benutzerinteraktion erfolgt hauptsächlich über einen prominenten **Befehls- und Eingabebereich**.

- Natürliche Sprachbefehle oder detaillierte Prompts werden hier eingegeben.
- Die UI unterstützt Mechanismen (z.B. @-Symbol), um spezifische Kontext-Snippets, Dateien oder Abschnitte aus dem "Shared Thread" oder der verbundenen IDE auszuwählen und explizit in den Prompt einzubinden (z.B. @Datei:src/main.py, @Selektion).

4.3. Ansatz für einen gemeinsamen "Thread" oder Kontext

Der Kern des Konzepts ist die Verwaltung eines **zentralen, UI-verwalteten gemeinsamen Kontexts**.

- **Inhalt:** Dieser Kontext umfasst ausgewählte Code-Snippets, Dateiverweise, Terminal-Ausgaben, manuelle Notizen, Chat-Historie mit den Agenten und andere relevante Projektdaten.
- **Konzept (Bevorzugt - Option B aus Referenz):** Die Frontend-UI agiert als "Kontext-Broker". Verbundene IDEs (via Erweiterung/Agent) synchronisieren ihren relevanten Zustand (Code-Änderungen, Selektion, Terminal-Output) mit der UI über WebSockets/SRPC. Die UI speichert und strukturiert diesen Kontext. Beim Senden eines Befehls wählt die UI die relevantesten Kontextteile aus und sendet sie zusammen mit dem Prompt an den Ziel-Agenten.
- **Vorteile (Option B):** Ermöglicht Echtzeit-Synchronisation, strukturierte Datenverwaltung, volle Kontrolle durch die UI. Passt zur Staged-Architektur.
- **Herausforderung (Option B):** Benötigt eine spezifische Implementierung oder Erweiterung in jeder zu unterstützenden IDE.

4.4. Bewertung der Machbarkeit, insbesondere im Hinblick auf Windsurf

- **Machbarkeit für Cursor:** Technisch realisierbar. Eine Staged-eigene Erweiterung für Cursor, die die VS Code Extension API nutzt und über

WebSockets/SRPC mit der Frontend-UI kommuniziert, kann die erforderliche Schnittstelle bereitstellen.

- **Machbarkeit für Windsurf: Derzeit nicht machbar.** Aufgrund der fehlenden dokumentierten externen Schnittstellen ist eine direkte, zuverlässige Integration in das vorgeschlagene UI-Konzept (insbesondere die UI-verwaltete Kommunikationsschicht für einen gemeinsamen Thread) **nicht möglich**. Lediglich sehr eingeschränkte Fallbacks (wie manuelle Dateisystem-Synchronisation – Option A aus Referenz) wären denkbar, würden aber nicht den Anforderungen eines nahtlosen "Shared Threads" gerecht.

5. Beantwortung der Nutzerfrage

Die Nutzerfrage war, wie ein Frontend-Tool beide IDEs (Cursor und Windsurf) ansprechen und Agenten in einem gemeinsamen Thread arbeiten lassen könnte.

Basierend auf der Analyse und dem Konzept:

- **Ansprechen von Cursor:** Ist technisch möglich durch Entwicklung einer dedizierten Stagewise-Erweiterung für Cursor, die über WebSockets/SRPC mit dem Frontend kommuniziert.
- **Ansprechen von Windsurf:** Ist mit den derzeit verfügbaren, dokumentierten Schnittstellen **nicht möglich**. Windsurf kann extern nicht zuverlässig programmatisch gesteuert werden.
- **Gemeinsamer Thread über beide IDEs hinweg:**
 - Ein echter, **nahtloser und synchronisierter** gemeinsamer Thread (Kontext), wie im bevorzugten Konzept (UI-verwaltete Kommunikationsschicht) beschrieben, ist **nur für unterstützte IDEs (aktuell: Cursor)** realisierbar.
 - Eine gleichzeitige Verwaltung eines solchen Threads, der Code und Zustand aus **sowohl Cursor als auch Windsurf** in Echtzeit integriert, ist aufgrund der technischen Limitationen von Windsurf derzeit **nicht machbar**.
 - Die einzige theoretisch mögliche Methode, Kontext zwischen einer UI und Windsurf auszutauschen, basiert auf den Erkenntnissen der Analyse, ist ein rudimentärer **Dateisystem-basierter Austausch** (Kontext als Dateien speichern/laden), der jedoch keinen nahtlosen oder synchronisierten "Thread" im Sinne des Konzepts darstellt.

6. Fazit und Ausblick

Die Entwicklung einer zentralen Frontend-UI zur Steuerung von Code-Agenten und zur Verwaltung eines gemeinsamen Kontexts ist ein vielversprechender Ansatz, der zur Architektur von Systemen wie Stagewise passt.

- **Machbarkeit:** Das Konzept ist technisch **realisierbar für IDEs, die eine geeignete Schnittstelle bieten**, insbesondere für Cursor durch die Entwicklung einer spezifischen VS Code-kompatiblen Erweiterung/Agenten, die über WebSockets/SRPC mit dem Frontend kommuniziert.
- **Limitationen:** Die Integration von Windsurf ist aufgrund der fehlenden dokumentierten externen Schnittstellen derzeit die **wesentliche Einschränkung**. Ein echtes "gemeinsames Thread"-Erlebnis über Windsurf hinweg ist mit den aktuellen Mitteln nicht umsetzbar.
- **Empfehlung:** Die Weiterentwicklung sollte sich zunächst auf die Implementierung des Konzepts für unterstützte IDEs wie Cursor konzentrieren. Parallel dazu sollte die Entwicklung von Windsurf aufmerksam verfolgt werden, um festzustellen, ob zukünftig eine öffentliche API oder andere Integrationsmöglichkeiten bereitgestellt werden. Die Definition eines standardisierten Protokolls zwischen einer solchen UI und IDE-Agenten/Erweiterungen könnte zukünftige Integrationen erleichtern.